DAY 1

PAGE No. / DATE / /

Task 3 Loop Complexity

Dry run
Initially.
$i = 0$

Check $i < 3$
$0 < 3 \rightarrow$ True
Print 0
$i++ \rightarrow i = 1$

Check $1 < 3$
$1 < 3 \rightarrow$ True
Print 1
$i++ \rightarrow i = 2$

Check $2 < 3$
$2 < 3 \rightarrow$ True
Print 2
$i++ \rightarrow i = 3$

Check $3 < 3$
$3 < 3 \rightarrow$ False

Loop ends.

- single()
  Big O = O(n)

Dry run
$n = 5$
$c = 0$

$i = 0$
$0 < 5 \rightarrow$ True
$c++ \rightarrow c = 1$
$i++ \rightarrow i = 1$

$i = 1$
$1 < 5 \rightarrow$ True
$c++ \rightarrow c = 2$
$i++ \rightarrow i = 2$

$i = 2$
$2 < 5 \rightarrow$ True
$c++ \rightarrow c = 3$
$i++ \rightarrow i = 3$

$i = 3$
$3 < 5 \rightarrow$ True
$c++ \rightarrow c = 4$
$i++ \rightarrow i = 4$

$i = 4$
$4 < 5 \rightarrow$ True
$c++ \rightarrow c = 5$
$i++ \rightarrow i = 5$

i = 5
5 < 5 → false
Loop ends

return c
return 5

- nested()
Big O = $O(n^2)$

n = 3
c = 0

i = 0
j = 0
0 < 3 → true
c = 1

j = 1
1 < 3 → true
c = 2

j = 2
2 < 3 → true
c = 3

j = 3
3 < 3 → false

i = 1
j = 0 → c = 4
j = 1 → c = 5
j = 2 → c = 6

i = 2
j = 0 → c = 7
j = 1 → c = 8
j = 2 → c = 9

i = 3
3 < 3 → false

n = 3
c = 9

3 × 3 = 9

- triangle()

n = 4
c = 0

PAGE No.
DATE / /

i = 0
j < 0
No execution
c = 0

i = 1
j = 0
0 < 1 → True
c = 1

j = 1
1 < 1 → False

i = 3
j = 0 → c = 4
j = 1 → c = 5
j = 2 → c = 6
j = 3 → false
return 6.

i = 2
j = 0
0 < 2 → true
c = 2

j = 1
1 < 2 → True
c = 3

j = 2
2 < 2 → false

- halving()
Big O = O(log n)
n = 16
c = 0

i = 16
16 > 1 → True
c = 1
i = 8

8 > 1 → true
c = 2
i = 4

4 > 1 → true
c = 3
i = 2

2 > 1 → true
c = 4
i = 1

1 > 1 → false

return 4

PAGE No.
DATE / /

- nlogn()

Big O = O(n log n)

j = 1	j = 2	j = 4
1 < 8 → True	2 < 8 → True	4 < 8 → True
c++	c++	c++

j = 8
8 < 8 → false.

- SqrtLoop()

p = 1	p = 2
1 x 1 <= 16 → true	2 x 2 <= 16 → True
c = 1	c = 2

p = 3	p = 4
3 x 3 <= 16 → True	4 x 4 <= 16 → True
c = 3	c = 4

p = 5
5 x 5 <= 16 → False

Recursion Cost

Big O = O(n)

- factorial()

calls = 0

PAGE No.
DATE / /

Call 1
factorial(4)
calls = 1
4 <= 1 → False
return 4 * factorial(3)

Call 2
factorial(3)
calls = 2
3 <= 1 → False
return 3 x factorial(2)

Calls 3
factorial(2)
calls = 3
2 <= 1 → false
return 2 x factorial(1)

Call 4
factorial(1)
calls = 4
1 <= 1 → True
return 1

Now return
factorial(2) = 2 x 1 = 2
factorial(3) = 3 x 2 = 6
factorial(4) = 4 x 6 = 24

Final: factorial(4) = 24, calls = 4

- binarySearch()

Big O = O(n)

Suppose: a = [10, 20, 30, 40, 50, 60, 70]
Key = 60

Initially
lo = 0
hi = 6

Middle
mid = 0 + (6-0)/2
mid = 3
a[3] = 40
60 > 40, search right.

lo = 4
hi = 6
mid = 5
a[5] = 60
return 5.

PAGE No.
DATE / /

Growth Experiment

~~bash~~

timeMs().

n = 5000

array of size 5000

Call linear(a)
→ Start timer
→ execute linear loop
→ Stop timer
→ Store t1

Call nlogn(a)
→ Start timer
→ Copy array
→ Sort array
→ Stop timer
→ Store t2

Call quadratic(a)
→ Start timer
→ execute nested loops
→ Stop timer
→ Store t3

Print n, t1, t2, t

Duplicate Check

- hasDuplicateNested()

Suppose : a = [10, 20, 30, 40]

i = 0
a[i] = 10

j = 1
10 == 20 → false

j = 2
10 == 30 → false

j = 3
10 == 20 → False.

Next
i = 1
a[i] = 20

j = 2
20 == 30 → false

j = 3
20 == 20 → ~~false~~ true

- hasDuplicateSet()

Suppose: [10, 20, 30, 20]

seen = []

x = 10
add 10 → True
seen = [10]

x = 20
add 20 → True
seen = [10, 20]

x = 30
add 30 → True
seen = [10, 20, 30]

x = 20
add 20 → false
Duplicate found
return true.

String Concat Trap

For n = 4
Initially: S = " "

Iteration 1:
S = " " + 'x'
S = "x"

Iteration 2:
S = "x" + 'x'
S = "xx"

Iteration 3:
S = "xx" + 'x'
S = "xxx"

Iteration 4
S = "xxx" + 'x'
S = "xxxx"

PAGE No.
DATE / /

List Traps

Start: [10, 20, 30, 40]

Remove index 0:
remove 10
shift 20
shift 30
shift 40
[20, 30, 40]

Again
remove 30
shift 40
[40]

Again
remove 40
[]

Remove index 1
remove 20
shift 30
shift 40
[30, 40]

Linear Search Count

For a = [0, 3, 6, 9, 12] Key = 9

comparison = 0

i = 0
a[0] = 0
0 == 9 → false
comparison = 1

i = 1
3 == 9 → false
comparison = 2

i = 2
6 == 9 → false
comparison = 3

i = 3
9 == 9 → true
comparison = 4

return 3.

PAGE No.
DATE / /

Second Largest

Input : 12, 35, 1, 10, 34, 1

Sorted: 1, 1, 10, 12, 34, 35

Largest = 35

Start from index 34
34 | = 35
return 34.

Program To Interface

list = [10, 20, 30]

Index loop

i = 0	i = 1	i = 2
get(0) = 10	get(1) = 20	get(2) = 30
s = 10	s = 30	s = 60

StackAdt Demo

I/P : "ABC"
Stack = empty

push A
Stack = [A]

push B
Stack = [A, B]

push C
Stack = [A, B, C]

pop C → "C"
Stack = [A, B]

pop B
result → "CB"

pop A
result = "CBA"

Stack empty.

return "CBA".

PAGE No.
DATE / /

JavaStack Flow

Initially: []

push A → [A]
push B → [A,B]
push C → [A,B,C]

add at index 0
[SNEAKY, A, B, C]

0 → SNEAKY.
1 → A
2 → B
3 → C

Remove B
Result: [SNEAKY, ~~B~~ A, C].